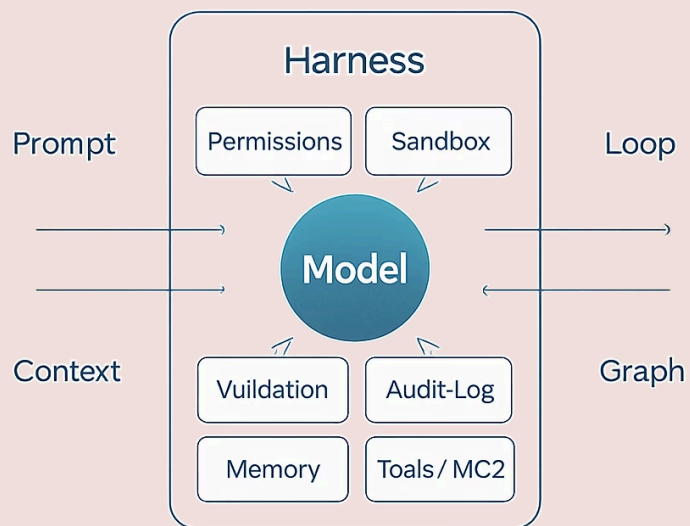


控制权往哪里移：AI Coding 的工程演变

从写提示词到管运行边界——不只是写代码，是所有 AI 交付的质量问题

Harness 的构成与证据·边界与工具选择·交付质量怎么保证

Ethan Jiang



从 Coding 切入，但问题不只属于工程师

背后是同一个问题：交付质量

AI agent 现在在做什么

- 写代码
- 生成内容 / 发布文章
- 产出报告
- 剪辑视频

交付质量，怎么保证？

- 怎么保证它持续做对、产出质量稳定？
- 做错了能不能查——可追溯
- 长期不越跑越偏——可治理
- 谁说了算——权责清晰

 Coding 只是第一个把这件事逼到台面上的场景——因为代码出错的代价最直接

每个术语背后，都是行业榨出 LLM 能力时的一次摸索

这几年冒出来的新名词，不是随机的——每一个都有它出现的理由

控制权一步步往模型外面移：

- 最早：管它说什么
- 后来：管它能看到什么
- 再后来：管它能不能动、动了谁负责

"I feel a lot of things changing as working software increasingly comes out on a tap."

— — Andrej Karpathy, 2026年6月（论 AI 生成软件的演变）

五类做法依次出现，控制位置越来越往模型外面移 越往后，管的不只是它说什么——而是它能不能动

1

Prompt Engineering

最早：把规矩写进话里

2022年随ChatGPT普及

2

Context Engineering

接着：把事实塞进上下文

2025年6月，Tobi Lütke & Karpathy 命名

3

Harness Engineering

然后：把模型圈进受控环境

2026年2月 Mitchell Hashimoto 提出，4月 Birgitta Böckeler 系统化

4

Loop Engineering

再后：让多轮之间互相影响

2026年6月，Addy Osmani 命名

5

Graph Engineering

最新：把多个 agent 串成一张图

2026年7月，Peter Steinberger 一条推文引爆

前两类管生成，后三类才管能不能发生

这是五类之间最重要的分界

① Prompt · ② Context

管的是：模型说什么

- 输出可以更好
- 但这一步能不能发生——管不了

③ Harness 开始

管的是：执行本身

- 控制对象变了——不是输出
- 很多团队把这两件事混在一起，是最常见的误区

⚠ 管生成 ≠ 管执行——这是最容易踩的坑

① Prompt Engineering

把规矩写进话里

前提是模型会照做——现在这个前提越来越不成立了

两年前

三小时调措辞，效果立竿见影

现在

调不调，差别已经不大了

② 它还在，但不是杠杆了——别再把时间花在调措辞上

"The inferred definition of 'prompt engineering' is that it's a laughably pretentious term for typing things into a chatbot. That inferred definition is the one that stuck."

—— Simon Willison, simonwillison.net, 2025年6月

② Context Engineering

把事实放进上下文

前提是放进去它真的用得到——最好的上下文工程往往是少放，不是多放

两个反直觉的事实：

这一类改的是它能看到什么，不是能不能上线——这是这一类的天花板。

- 上下文太长，模型对中段的召回会下降
- 最小脚手架（一个提示词 + Bash + Edit）照样跑得出结果

所以：少放，不是多放。

"Context engineering is the delicate art and science of filling the context window with just the right information for the next step."

— — Andrej Karpathy, 2025年6月

③ Harness Engineering

Harness 管的是「能不能发生」

前两类只改模型说什么——改不了这一步能不能发生

前两类管不了什么

- 权限——谁能做这一步
- 沙箱——在哪里做
- 验证门禁——做对了吗
- 执行记录——做了什么

Harness 管的就是这些

这一圈叫 **harness**；这一类叫 **Harness Engineering**

⚠ 没有 harness，谈执行控制就是空话

"Coding agents also provide us, their users, with many features to build an outer harness specifically for our use case and system."

—— Birgitta Böckeler, 《Harness engineering for coding agent users》, martinfowler.com, 2026年4月

④ Loop Engineering

让上一轮的结果改变下一轮

前提是单次运行受控——不然循环只是把错误复制得更快

四要素

触发 · 验证 · 停止条件 · 记忆

边界

先有受控的单次运行，才有可靠的循环

"I don't prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops."

—— Boris Cherny (Claude Code 负责人, Anthropic), 2026年6月

⑤ Graph Engineering

多个 agent 之间的编排

前提是单个 agent 受控——不然编排只是把错误放大

DAG 结构

显式节点 / 显式边 / 显式状态

图可以编排循环，也可以编排工具与验证器 —— 不要把两种图画成同一条线

边界与成熟度

每个环节各自失控时，编排只是把错误放大。

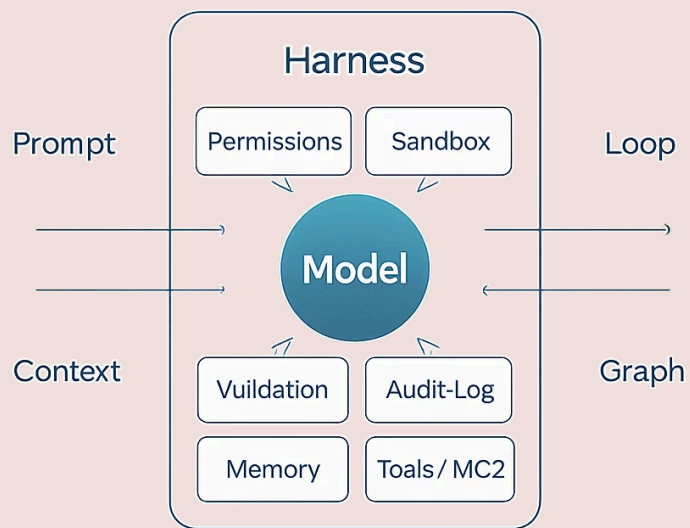
成熟度：这一类与 Loop 都是方向性观察，不是既定范式。

"Are we still talking loops or did we shift to graphs yet?"

—— Peter Steinberger (OpenClaw 作者)，X，2026年7月17日（获得 290 万次浏览）

五类看下来，终点始终是那一圈

Agent = Model + Harness



- 左边两支箭头 (**Prompt · Context**)：管模型说什么、能看到什么——但管不了这一步能不能发生
- 中间那一圈 (**Harness**)：权限·沙箱·验证·执行·记忆——这才是真正的控制边界
- 右边长出来的 (**Loop · Graph**)：建在 Harness 之上，没有受控的 Harness，它们只是把错误放大

"The term harness has emerged as a shorthand to mean everything in an AI agent except the model itself — Agent = Model + Harness."

—— Birgitta Böckeler, martinoflower.com, 2026年4月

Harness 在模型外面做了九件事

模型只管输出文字——这九件事，模型一件都做不了

执行循环

决定什么时候再问模型一次

工具集

文件 / 终端 / 浏览器 / MCP，它能调用什么

执行环境

它真正跑在哪：本地 / 容器 / 云沙箱

上下文管理

决定模型看到什么、压缩什么、丢掉什么

状态与记忆

跨轮次保存什么、崩溃后恢复什么

反馈与验证

测试 / lint / schema 判断这一步对不对

安全控制

权限 · 门禁 · 拦截 · 审批挂起

编排

子 agent 怎么拆、怎么汇、怎么路由

扩展接口

插件 / skills / 钩子，让外部能改它

 模型是大脑，这九件事是让大脑能干活的身体

同一个模型，harness 不同，交付质量就不同

不同团队用同一个模型，差距不在模型里

同一个模型 · 同一个版本

差距体现在三个维度：

- **成功率** —— 任务能不能跑完
- **成本** —— token 消耗和重试次数
- **可追溯性** —— 出了问题能不能查

差的不在模型里

谁放行、能碰什么、留下什么记录 —— 这三件事，恰好就是 harness 负责的。

① 同一个模型，换一圈 harness，就是两套交付系统

换的是那一圈，不是模型

同一个模型，harness 开源后：13.3% $\Rightarrow$ 38.3%

发生了什么

- 2026-08-20，Codex 开源其 harness (Apache-2.0)
- 开源的不是模型 — 是模型外面那一圈

同一个模型，结果

- ARC-AGI-3: 13.3% $\rightarrow$ **38.3%**
- 输出 token: 减少到原来的 **1/6**

 同一个模型，harness 不同，结果差了将近三倍 — 能力看模型，差距看那一圈

"Harness design can materially change results: on ARC-AGI-3, retained reasoning and context compaction raised GPT-5.6 Sol's score from 13.3% to 38.3% while reducing output tokens sixfold."

— OpenAI 官方开发者博客，《Codex as a platform: build on the open agent harness》，2026年8月19日

Harness 分层效应真实存在

两项独立实测，同一结论：harness 不同，结果相差数倍

实验一：Claw-SWE-Bench

同一个 GLM 5.1 模型

- minimal adapter: 19.1%
- 完整 adapter: **73.4%**
- 相差近 4 倍

实验二：受控上下文实验

169 个任务，只改上下文管理策略

- 完整解决: 43 → **72**
- 不换模型，不换任务，只换那一圈

① 两项独立实测，同一结论：差距在那一圈，不在模型

模型没换、代码没换、团队没换
差距在那一圈。

那一圈，该由谁来定？

TOOL CHOICE

先问一个问题：你的产物是什么？

这个答案，决定了你需要什么样的 harness

传统程序：

- 行为在写代码时就定了
- 输出可以穷举、测试可以覆盖
- 出了问题查代码路径
- 责任在写代码的人

智能体：

- 行为在运行时才被决定
- 输出不可穷举，每次跑都可能不同
- 出了问题查的是"那一次运行"
- 责任在写代码的人 + 让它跑起来的那一圈

你在做哪种，决定了接下来怎么选工具。

TOOL CHOICE

用 AI 写传统程序——现成工具够用

够用的原因不是工具好，是模型真的懂传统程序怎么写

训练材料极其丰富

几十年的开源代码、RFC、设计模式、最佳实践——模型全见过。

它真的懂怎么写好

TDD、spec-driven development、重构——它知道什么叫好代码，不只是能跑的代码。

现成工具直接用就好

Codex / Claude Code 写出来的传统程序质量本身就高，直接用就行。

TOOL CHOICE

用 AI 写智能体——代码没问题，harness 设计得人来不是 AI 不行，是好的 harness 长什么样，它还没见过足够多

写业务代码：没问题，直接用智能体的逻辑、工具调用、测试——AI 写起来和普通程序没区别。

harness 设计是另一回事：架构决策，不是实现细节

停止条件怎么设、工具权限边界在哪、出了问题怎么回溯——这些定错了，智能体跑起来就失控。

为什么 AI 在这里不擅长：训练材料太少

传统程序有几十年积累，但“好的 harness 长什么样”，2026 年才刚开始有论文研究。

TOOL CHOICE

工具很多，重点看两个把控制权还给你的 大多数工具把那一圈的边界锁在厂商手里

现成工具：能力强，但边界锁死

Claude Code · Codex · 各家模型公司自建 —— 上手快、能力强。但那一圈的边界由厂商定：用哪个模型、执行后端在哪、能碰什么 —— 你改不动。

如果你的产物本身是 **agent**，这件事就不能将就

传统程序用现成工具没问题 —— 厂商帮你把那一圈内置好了。但如果你在构建 **agent**，**harness** 的边界就是产品的一部分。边界在厂商手里，产品的核心就在厂商手里。

接下来看两条路的具体差异 —— 边界在谁手里，是唯一的分界。

边界在谁手里，是唯一的分界

两条路，本质区别只有一个

	现成工具（Codex / Claude Code）	自建 Harness（Pi / DSH）
边界由谁定	工具厂商	你自己
治理内置了什么	沙箱·审批·事件流，够用	你决定装什么、怎么装
改得动吗	模型 / 工具 / 执行后端，基本改不动	三层 seam 全部可换
适合什么场景	写传统程序、任务在能力圈内	产物本身是 agent，边界是产品的一部分

Pi 和 DSH 是同一类东西——都是把 Harness 交回你手里。区别只在定制深度，不是两条不同的路。

自建这条路：Pi 是最轻的那个选择

Pi：把 harness 的接缝交回你手里

执行后端可换

本地 shell / 容器 / 云沙箱，三种后端随时切换；权限边界由你定，不是厂商定。

工具集可换

15+ 模型供应商，mid-session 切换不中断任务。

行为组合可换

树状会话历史，任意分支恢复；extensions 全面开放，按需插拔。

GitHub 72,970 stars · MIT · 2025-08-09 开源

 一年以上社区打磨，72,970 stars，稳定可用——如果你想要的是轻量、可直接上手的自建路径，Pi 够了。想要更深的控制权和更先进的架构理念，看下一个。

自建这条路：DSH 是控制权最完整的那个选择

2026年新出，生态还在成形——但它引发的关注和它背后的架构理念，已经超出了一个“新工具”该有的分量。

DSH 的所有能力都以插件形式挂载 包括改它自己——没有必须动源码才能改的东西

插件化架构意味着

- 不存在必须改源码才能动的东西
- 所有能力以插件形式挂载
- 包括「改它自己」这件事

四种运行模式

本质是四套默认插件集

- 不是四种产品
- 可以按需拆开、按需组合

插件位不是设计稿——社区已经在批量供给那一圈的零件，下一张看具体分布。

来源：源码级一手研究；生态数字 ⚠️ 趋势级

⚠️ 稚嫩但不幼稚——理念领先，成熟度还在追赶。适合愿意押注前沿、自己能把控风险的团队。

DSH 的审计能力

会话日志：审计是架构自带的，不是事后补的 同一份日志，四种用法——出了问题，每一步都有据可查

回放

还原那一次运行的完整过程
—— 每一步按原序重现，分毫不差

恢复

从任意一步重新开始——不必从头跑，直接接在断点之后

分叉

在某一步产生两个不同的执行路径——同一前提，两种走法并行比较

审计

出了问题，每一步都有据可查。

❗ append-only 事件日志不是功能——是治理的地基。没有它，可追溯就是空话。

社区在动哪里——插件分布揭示真实定制热点

最热的不是模型层——是会话、工具和流程那一圈

529条

界面与外观

占比 23% | 给聊天窗做壳：侧栏 / 皮肤 / 桌宠

387条

Agent 能力扩展

占比 17% | 给 agent 添加工具和技能包

268条

会话·记忆·上下文

占比 12% | 让 agent 记得住、接得上

247条

开发者 workflow

占比 11% | 把 harness 变成 IDE / DevOps 工位

220条

渠道与触达

占比 10% | 推到 IM、连回家里、操控浏览器

安全散布在每一类里——治理不是单独一层，是**每个定制点都要考虑的事**。

社区在批量供给零件——DSH 自己是怎么把这套东西组织好的？

MAKING IT WORK · DSH

DSH 的工程立场 · 为 coding agent 设计的仓库

不赌聪明，赌成本结构

让"做对"成为阻力最小的路径，让"做错"在最近的地方撞墙

三个检验问题

知识外置了吗？正确入口明确吗？错误何时被发现？

两类失败，六个缺口

糊涂：铁律没外置 | 乱发挥：改动落点不明

做法：把知识和规则都接到机器上

知识写进可搜索的文件；规则接到能失败的命令

📖 接下来四页，依次看：结构 → 入口 → 路径与反馈 → 运行时落地

DSH 项目结构 · 每类事实只住在负责它的那一层

一个事实，一个 owner

每类事实只住在负责它的那一层，别处只放链接

tier taxonomy (谁说什么话)

四层各有分工：根 AGENTS.md 放常驻规则，子树放专属规则，README 放当前合同，Skill 放可复用流程。

当前状态和决策理由，分开放

docs/ 只写系统现在怎样；ADR 写为什么这样决定。两者分开，agent 才能区分现状和历史。

"不做什么"也要记下来

把已知限制、被否决的方案写进文件——agent 就不会反复提出同样的建议。

可读性 = 找得快，而不是文件少

能快速找到那份最小且权威的材料，并知道下一层该读什么。

① 一个事实一个家，其余地方只 link —— 先消灭"同一条规则在多处各写一版、然后互相漂移"。

DSH 入口链 · fresh agent 进来先读什么

fresh agent 进来，先看到什么

入口不是一份大文档，而是一条路——每多走一步才付那一步的上下文成本

层级	作用	字数上限	校验
根 AGENTS.md	常驻规则 · 仓库布局 · 命令表，详细内容 link 出去	≤ 1,950 词	verify-doc-budgets
子树 AGENTS.md	有专属规则的子树才放，路由层负责指路	≤ 750 词	verify-doc-budgets
docs/AGENTS.md	文档区专属规则	≤ 1,320 词	verify-doc-budgets
各 README.md	每个包的当前合同，具体事实住在这里	≤ 994 词	verify-doc-budgets

 文件态先做好：短根指令 + symlink + 按需放子树指令 + 字数预算。这是写文件的工程，收益最高、成本最低。

DSH 的两道闸 · 防住乱发挥的工程机制

两道闸，把执行边界锁住

先知道该改哪里，再让机器验证改得对不对

闸一 · 正确路径（先问归属，再谈实现）

- 先确认这个功能属于哪一类，再决定在哪里动手
- 新行为挂到已成文的扩展点；改核心循环是最后的选项

闸二 · 可执行反馈（做错了早被发现）

- 六层反馈各证明一件事：编译 → 解析 → 局部测试 → 快照 → 运行时检查 → 语义 review
- 每条可机械判断的规则，配一个 `exit non-zero` 的命令

④ 规则接到执行，agent 做错了立刻被机器拦住 —— 这是最高杠杆的一步。

DSH 运行时 · 按成本顺序落地

披露分层，按成本顺序搬 从最高收益开始，按顺序来

01

Phase 0-1 · 立 owner（收益最高）

用三问给项目打分 → 短根 AGENTS.md +
symlink + 事实 → home 表

02

Phase 2 · 分开"现在"和"为什么"

含负知识，决策理由进 docs/adr/

03

Phase 3 · 铺正确路径

归属表 + 2-3 份 cookbook

04

Phase 4 · 接可执行反馈（回报第二高）

把可机械规则接到执行，每条做一次负例控制

05

Phase 5-7 · Skills + 运行时查询 + 防漂移

留到真有组合压力的时候 — 提前造会让维护成本吃掉可读性收益

① 一个周末的 MVP：短 AGENTS.md + symlink；决策理由进 docs/adr/；一张归属表；2-3 条规则接成 exit non-zero。这四件事覆盖八成收益，不需要任何插件架构。

MAKING IT WORK

Harness 建好了，产出是什么不重要

可靠性的逻辑只有一套——不因产出不同而改变

coding agent 写代码

权限、沙箱、门禁、日志——这套建好了，代码才能进生产

agent 写文章·生成内容

同一套问题：谁批了这一步？在哪里跑？出了问题能追吗？

agent 跑工作流·产出报告

执行边界没有定义，工作流就是一个放大错误的循环

传统程序 + AI 辅助

AI 写的代码进了传统程序，harness 决定它能不能被信任

④ 同一套治理逻辑，不因产出不同而改变——建好了，就是护城河。

模型不是瓶颈，Harness 才是控制权在哪里，交付质量就在哪里。

这几年每个新名词，都是同一件事往前走了一步：

- Prompt → 管它说什么
- Context → 管它能看到什么
- Harness → 管它能不能动
- Loop → 管多轮之间怎么跑
- Graph → 管多个 agent 怎么协作

"When an agent misbehaves, the fix is usually code in the harness, not a better prompt. That practice is called harness engineering."

— — Tejas Kumar, dev.to, 2026年9月

模型会继续变强 — 但能回答这四个问题的团队，才是真正拿住了那一圈的人。